

A Blockchain-Based Anti-Money Laundering Fraud Detection System Using Machine Learning

1st Mayesha Marzia Zaman

Dept. of Computer Science and Engineering
Khulna University of Engineering & Technology
Khulna, Bangladesh
zaman2007088@stud.kuet.ac.bd

2nd Nabil Faiyaz Sadi

Dept. of Computer Science and Engineering
Khulna University of Engineering & Technology
Khulna, Bangladesh
nabil@cse.kuet.ac.bd

Abstract—Blockchain-based anti-money laundering (AML) systems anchor fraud alerts on a shared ledger to make them tamper evident. This leaves a gap: a bank that wishes to conceal a transaction need not tamper with the ledger, it need only decline to raise an alert, and the ledger faithfully records nothing. Anchoring a receipt for every scored transaction makes each receipt immutable but still does not make the receipt set verifiable. We present a machine learning AML platform on Hyperledger Fabric that closes this gap. A per-organization counter in world state assigns each processing receipt a sequence number inside the same endorsed and ordered transaction that writes the receipt, with the submitting organization resolved from the transaction creator certificate rather than from an argument. Because the counter advances only on commit, the committed series is gap free by construction, so a missing receipt becomes an arithmetic shortfall the regulator computes from its own peer replica. Confidentiality on a ledger shared with a competing institution is preserved by keyed pseudonymization and a private data collection scoped to the bank and the regulator. On the Elliptic dataset under a temporal split, the deployed LightGBM detector reaches 65.2% recall at 0.963 ROC-AUC, and the audit layer runs entirely off the scoring path.

Index Terms—anti-money laundering, Hyperledger Fabric, machine learning, audit completeness, tamper-evident logging, private data collections, financial fraud detection

I. Introduction

Anti-money laundering (AML) compliance absorbs a large and growing share of bank operating cost yet remains largely ineffective. Conventional know your customer (KYC) and AML programmes rest on centralized databases and expert authored rule engines that trigger on static thresholds and country risk lists. Such systems do not adapt to new laundering typologies, and practitioners report false positive rates so high that the bulk of analyst capacity is spent dismissing legitimate activity [1].

Supervised machine learning addresses the detection half of this problem well. Tree ensembles and graph neural networks trained on labelled transaction graphs outperform rule engines on benchmark data [2]–[4]. What machine learning does not address is the trust half. When a model runs inside the institution under regulatory scrutiny, the model score, the alerting threshold, the decision to escalate, and the record that any of this happened are all controlled by the party being audited.

A regulator examining a relational audit log is examining a database whose administrator can update rows, disable triggers, and truncate write ahead logs.

Permissioned blockchain answers the second problem, and prior work proposes Hyperledger Fabric as a shared KYC registry and AML record store [7]–[9]. These proposals typically anchor alerts: transactions the bank has already decided are suspicious. A bank wishing to conceal a transaction need not tamper with the ledger; it simply never raises an alert, and the ledger, being a faithful record of what it was told, records nothing. Anchoring flagged transactions makes the flagged set immutable; it does nothing to make the unflagged set observable. We call this the fabrication gap.

The obvious repair is to anchor a receipt for every scored transaction. This is necessary but not sufficient, and the insufficiency is subtle. Anchoring every transaction makes each receipt immutable but does not make the set of receipts complete. If the bank never submits a receipt, or a fire and forget anchoring call fails silently, the ledger holds no evidence that anything is missing. The regulator can verify what exists but cannot bound what does not.

Detecting omission from a log is not itself new. Truncation attacks have been studied for two decades in the single-logger setting, where an untrusted host keeps a log and auditors challenge it [5], [6]. Our contribution is a construction that enforces completeness inside a multi-organization permissioned ledger, where the verifier is a consortium member holding its own replica rather than an external auditor issuing challenges, and where the same ledger is shared with a competing institution from which business data must be withheld. Specifically:

- Provable receipt completeness. The audit chaincode assigns each receipt a per-organization sequence number inside the same endorsed transaction that writes it, with the organization resolved from the creator certificate. The committed series is gap free by construction, converting a missing receipt from an invisible event into an arithmetic shortfall.
- Privacy preserving multi-party auditability. Customer identifiers are pseudonymized with a keyed

hash, and sensitive receipt fields move into a private data collection scoped to the bank and regulator, leaving only a SHA-256 digest on the shared record.

- A blockchain to model feedback loop. The alerting threshold is selected at inference time from the on-chain KYC risk tier, making the FATF risk based approach a governed input to the model decision boundary.
- An empirical evaluation of three tree ensembles on the Elliptic dataset under a temporal split, with an analysis of the precision recall tradeoff that motivates the risk based threshold.

II. Related Work

A. Machine learning for illicit transaction detection

Weber et al. released the Elliptic Bitcoin dataset and established that supervised tree ensembles detect illicit nodes competitively with graph convolutional networks [2]. Elmougy and Liu extended it with wallet-level actors [3], and Pareja et al. showed that evolving network parameters over time improves temporal classification on the same graph [4]. All of this work is centralized: the model output is a row in a database, with nothing binding it to an external verifier.

B. Tamper-evident logging and completeness

Proving a log has not been silently truncated predates blockchain. Crosby and Wallach gave the construction closest to our concern: a history tree letting an untrusted logger prove, with logarithmic proofs, that no event was removed [6]. More recent systems pursue the same guarantee by other means: SealFSv2 combines storage-based logging with key ratcheting so that entries written before a compromise cannot be counterfeited afterwards [5].

These assume a single logger and auditors who issue challenges, and establish completeness with respect to what the logger accepted. Our setting differs on three axes. The log is replicated across organizations, so verification is a local read at the regulator’s peer rather than an interactive proof. Completeness is enforced by chaincode at write time, inside consensus, rather than proven on demand. And the verifier is a consortium member, which introduces a confidentiality requirement the single-logger literature does not face.

C. Blockchain for KYC, AML, and auditability

Kang et al. built FabZK, concealing amounts and counterparties on Fabric with Pedersen commitments and zero-knowledge proofs while retaining auditability [7]; it is the closest prior work to our privacy construction, but targets transaction confidentiality rather than record-set completeness. Khan et al. combined Fabric with federated learning for AML compliance [8]. Fabric itself provides the channel, endorsement, and private data machinery these designs rest on [9]. Table I positions this work.

TABLE I
Accountability properties of representative prior work

System	Records all events	Set complete	Conf. from verifier	Multi-org ordering
Weber et al. [2]	—	—	—	—
Crosby–Wallach [6]	yes	yes	no	no
SealFSv2 [5]	yes	yes	no	no
FabZK [7]	no	no	yes	yes
BAML [8]	[[?]]	[[?]]	[[?]]	yes
This work	yes	yes	yes	yes

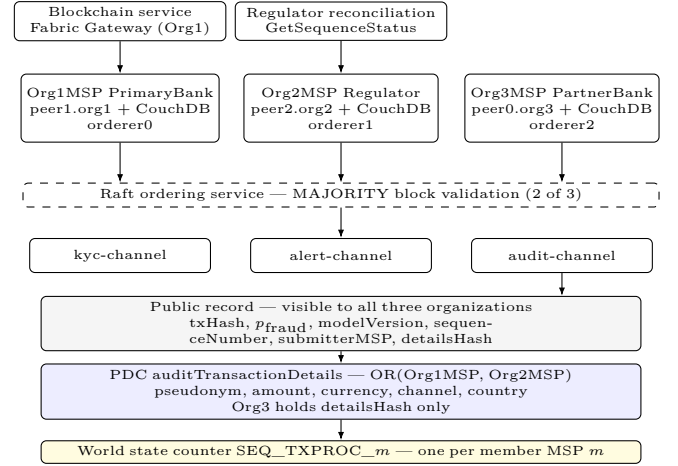


Fig. 1. Three-organization consortium. Each business organization runs one peer and one Raft orderer, so no ordering party sits outside the consortium. All three organizations are members of all three channels; confidentiality comes from the private data collection and pseudonymization of Section V, not from channel exclusion. The counter SEQ_TXPROC_m is the mechanism of Section IV.

III. System Architecture

A. Consortium, channels, and private data

The Fabric network is a three organization consortium (Fig. 1). Org1MSP is a primary bank that onboards customers, scores transactions, and files reports; Org2MSP is the regulator, performing oversight and completeness reconciliation; Org3MSP is a partner bank participating for cross-institution fraud signal.

Ordering uses a Raft service of three orderers, one per business organization. This departs from the common Fabric pattern of a separate ordering organization, because in an AML consortium that party would be a trusted third party with privileged control over what is committed. With one orderer per organization and a MAJORITY Writers block validation policy, at least two of three organizations sign every block. Blocks are cut on a two second batch timeout or twenty transactions.

The ledger is partitioned into three channels, each with its own Go chaincode: kyc-channel for identity records, alert-channel for fraud alerts, and audit-channel for processing receipts, model predictions, and report hashes. All three organizations are members of all three channels: the regulator has a mandate to observe every channel, and

the partner bank must see fraud signal for the consortium to be useful.

B. Services and transaction path

The platform comprises nine services behind an API gateway. Eight are Go services communicating over gRPC; the scoring service is Python. Relational records live in PostgreSQL, enriched transactions in MongoDB, velocity windows and cached risk tiers in Redis, with Kafka as the event backbone. Personally identifying fields are encrypted with AES-256-GCM via a Vault transit engine. The scoring decision is synchronous; the ledger anchor is asynchronous and never blocks the pipeline.

C. Feature extraction and domain adaptation

Models are trained on Elliptic, whose features are Bitcoin specific, while the deployment target is bank wire transfers. No public labelled bank transfer dataset of comparable scale exists, so Elliptic labels serve as a behavioral proxy and an explicit adaptation is defined. Of the 166 raw features, the 85 with highest LightGBM importance form the canonical input, shared by every model and by the gRPC contract. At inference on bank data, 38 of the 85 positions are populated from structured wire transfer fields and 47 are zero padded, encoding Bitcoin structural quantities such as unspent output statistics with no banking counterpart. The 38 mapped positions cover all six behavioral indicator categories of FATF Recommendation 16 [16]: value anomaly, velocity, geographic risk, network topology, temporal pattern, and customer profile. In institutional deployment the architecture would be retrained on institution specific labels, replacing the Elliptic weights entirely.

D. Risk based alerting as a governance loop

A single global threshold is inconsistent with the risk based approach FATF Recommendation 1 requires [16]. The threshold is instead a function of the KYC risk tier ℓ , written on the KYC channel by an authorized compliance officer and cached in Redis from the chain. An alert is raised when

$$p > \tau(\ell), \quad \tau : \{\text{LOW, MED, HIGH, CRIT}\} \rightarrow [0, 1] \quad (1)$$

with $\tau = 0.70, 0.55, 0.40, 0.25$ respectively: minimizing false positives for compliant customers, balanced, enhanced due diligence, and maximum recall. Detector sensitivity for a given customer is therefore not a constant inside the bank but a value governed on a multi-party ledger. When an officer raises a customer to CRITICAL, alerting behavior changes on the next transaction with no retraining, and the change is itself an immutable, attributed ledger record. Threshold manipulation, a classic route to suppressing alerts, becomes visible.

Algorithm 1 RecordTransactionProcessed with sequence assignment

```

1: input: public metadata  $R$ , private details  $D$  via transient map
2:  $m \leftarrow \text{MSPID}(\text{creator}(\text{stub}))$  {from certificate}
3:  $s \leftarrow \text{GetState}(\text{SEQ\_TXPROC\_}m) + 1$ 
4:  $R.\text{seq} \leftarrow s$ ;  $R.\text{submitterMSP} \leftarrow m$ ;  $R.\text{detailsHash} \leftarrow \text{SHA-256}(D)$ 
5:  $\text{PutState}(\text{SEQ\_TXPROC\_}m, s)$ 
6:  $\text{PutState}(\text{recordID}, R)$ 
7:  $\text{PutPrivateData}(\text{auditTxDetails}, \text{recordID}, D)$ 
8: emit AUDIT_RECORDED

```

IV. Provable Audit Completeness

A. Threat model

We assume the bank is rational and self interested rather than externally compromised: it will not forge endorsements or break SHA-256, but may wish a transaction never to have existed as far as the regulator is concerned. The regulator runs its own peer with a full replica and can obtain an independent count of the bank's processed volume, for example from regulatory returns. The partner bank is honest but curious: it participates in consensus and must not learn customer level detail. The required property is therefore not merely that anchored records are immutable, but that the anchored set is provably complete.

B. Chaincode assigned sequence numbers

The audit chaincode maintains a counter key $\text{SEQ_TXPROC_}m$ in world state per member MSP m . Algorithm 1 gives the receipt path. Two details carry the guarantee. First, the submitting identity m is read from the transaction creator certificate through the chaincode identity API, not from a function argument, so a submitter cannot attribute receipts to another organization. Second, the counter increment and the receipt write occur in one chaincode invocation and are therefore endorsed, ordered, and committed atomically, with the counter's read version validated at commit time by Fabric's multiversion concurrency control.

Let S_m be the latest committed sequence for organization m , returned by GetSequenceStatus . Because the counter advances only within committed transactions, receipts $1, \dots, S_m$ all exist; a gap cannot commit, because a transaction skipping a value would write a counter state inconsistent with its read version and be invalidated. Reconciliation is then the arithmetic

$$\Delta_m = N_m - S_m, \quad (2)$$

where N_m is the volume organization m reports independently (Fig. 2). Any $\Delta_m > 0$ is a provable shortfall. The bank cannot attribute it to lost messages, because a lost message would produce a gap and gaps cannot exist; the only consistent explanation is that the receipts

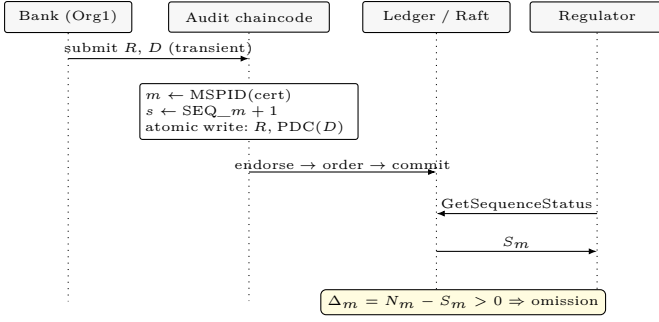


Fig. 2. Completeness protocol. The sequence number is assigned inside the endorsed transaction, so the committed series is gap free. The regulator reads S_m from its own replica and compares it with the independently reported volume N_m .

were never submitted. Backfilling is likewise detectable: a receipt anchored late receives a current sequence number while carrying a past processedAt timestamp, so skew between sequence order and timestamp order exposes retrospective insertion.

C. Concurrency cost

Every receipt from one organization reads and writes the same counter key, so concurrent submissions conflict under MVCC. The blockchain service serializes receipt submission with a mutex and retries validation codes 11 and 12 up to three times. At prototype throughput this is not the binding constraint. At production throughput the counter would need sharding, for example one counter per day, with (2) summed over shards. This is a real scalability limit of the mechanism and is stated as such.

V. Privacy Preserving Auditability

Because the audit and alert ledgers are replicated to a competing institution, no raw identifier or business field may appear on them in plaintext; an immutable ledger also admits no erasure, which data protection law expects.

A. Keyed pseudonymization

The blockchain service is the bank’s privacy boundary. Every customer identifier is rewritten before it leaves that boundary as

$$pseudonym = cust- \parallel HMAC\text{-}SHA256(k, customerID), \quad (3)$$

where k is held only by the issuing bank. The transformation is one way and is never undone; it is applied at the same boundary on both paths, to identifiers on the way in and query arguments on the way out, so upstream services keep using real identifiers while the ledger only ever observes pseudonyms. Because the mapping is deterministic, the regulator can still correlate all on-chain activity of one customer across channels, which is what an investigation requires. Because it is keyed rather than a plain hash, a channel member holding candidate identifiers cannot confirm any of them by recomputation, which a bare SHA-256 would permit.

B. Private data collection for receipt details

Pseudonymization protects identity but not business fields, so the receipt is split. The public record carries only fraud metadata — transaction hash, timestamp, fraud probability, risk level, alert flag, model version, sequence number, submitting MSP — plus the digest $detailsHash = SHA\text{-}256(D)$. The sensitive payload D (customer pseudonym, amount, currency, channel, country) is written to the collection auditTransactionDetails, whose policy is $OR(\text{Org1MSP.member}, \text{Org2MSP.member})$. The payload travels in Fabric’s transient field and never enters the public transaction payload, and endorsement is restricted to collection members, so the partner bank’s peer never receives it. The anchored digest nevertheless lets the regulator prove the private data was not altered afterwards: GetReceiptDetails re-hashes the retrieved payload and returns a verification flag. The resulting split is that all three organizations see the fraud metadata, the sequence number, and detailsHash, the bank and regulator additionally see the pseudonym, amount, and corridor, and only the bank can resolve a pseudonym to a natural person — off-chain, and for the regulator on lawful request. Suspicious activity reports follow the same pattern: the document is stored off-chain and only its SHA-256 digest and object key are anchored at filing time.

VI. Evaluation

A. Dataset and protocol

Experiments use the Elliptic Bitcoin dataset [2]: 203,769 transactions, 234,355 edges, 166 features per node, of which 94 are local and 72 aggregated from the one hop neighbourhood. Of these, 46,564 nodes are labelled — 4,545 illicit and 42,019 licit; unlabelled nodes were excluded.

The split is temporal, not random: the earliest time steps form the training set and the latest the test set [[state the exact boundary, e.g. steps 1–39 train / 40–49 test]]. A random split leaks future information through the graph neighbourhood and inflates reported performance; a temporal split reproduces the deployment condition. Class imbalance was addressed with SMOTE [13] on the training partition only. The test partition holds 9,643 labelled transactions, 503 of them illicit, a prevalence of 5.22%.

B. Models and metrics

Three tree ensembles were trained: Random Forest [10] (500 trees, depth 12, balanced class weights); XGBoost [11] (500 estimators, learning rate 0.05, positive class weight 9); and LightGBM [12] (1000 leaves, learning rate 0.05, unbalanced handling). Tree ensembles were chosen over deep alternatives because they tolerate strong class imbalance, train in seconds rather than minutes — which matters for the periodic retraining a production AML programme requires — and admit exact tree based attribution, so every escalated alert is explainable without a surrogate

model. Every prediction carries a SHAP attribution [14] serialized alongside it.

Because the positive class is rare, accuracy is near uninformative and is reported only for comparability. We additionally report the Matthews correlation coefficient

$$\text{MCC} = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP+FP)(TP+FN)(TN+FP)(TN+FN)}}, \quad (4)$$

which stays meaningful under imbalance because it uses all four cells of the confusion matrix [15].

C. Results

Table II reports performance at threshold 0.5, with the confusion counts from which accuracy and MCC are derived.

The three ensembles are close in ranking quality, with ROC-AUC between 0.955 and 0.964, but differ sharply in where they place the operating point. Random Forest is precision oriented: it raises only 42 false positives but misses 219 of 503 illicit transactions. LightGBM is recall oriented: it recovers 328 at the cost of 161 false positives, with XGBoost between them at similar precision but lower recall. Because the models agree closely on ranking, most of this difference is threshold placement rather than discriminative power — which is precisely why the system does not hard code a threshold.

Two comparisons deserve statement. First, against the published baseline: Weber et al. report illicit-class F_1 of 0.788 for Random Forest on all 166 features and 0.694 on the 94 local features alone [2]. Our Random Forest reaches 0.685, close to their local-feature result and materially below their full-feature result. The likely cause is the selection of Section III-C: our 85 dimensional input retains 68 local and only 17 aggregated features, discarding most of the one hop neighbourhood signal behind their gain. We report this openly because it bounds what the detection layer contributes; recovering the aggregate features is straightforward future work and is orthogonal to the accountability mechanism. Second, in absolute terms, even the best detector leaves 175 of 503 illicit transactions unflagged. This is the ordinary condition of AML detection, and it is exactly why the accountability layer matters: a system missing roughly a third of illicit activity cannot rest its credibility on detection quality alone, and must instead demonstrate that everything it did score was recorded faithfully and completely.

Which operating point is preferable is a compliance judgement, not a machine learning one. A false negative costs a regulatory penalty and an open laundering channel; a false positive costs analyst time. Under any plausible cost ratio the recall oriented point dominates, so LightGBM was selected as the production scorer, and it is also the cheapest to train. The system still expresses the tradeoff per customer through (1), using the same model at a precision oriented point for low risk customers and a recall oriented one for critical risk customers.

D. Completeness detection

To test whether the mechanism delivers the claimed property, we drive the audit chaincode against Fabric’s mock ledger. The bank processes $N = 1000$ transactions and suppresses a randomly chosen subset of k of them, submitting no receipt at all. This is the strongest form of the fabrication gap, because nothing is tampered with and no record is altered. The regulator then reads S_m through GetSequenceStatus from its own replica, without cooperation from the bank, and evaluates (2). Table III reports the outcome.

Three properties hold across every configuration. Reconciliation recovers the suppressed count exactly, so $\Delta_m = k$ throughout, and a single omission in one thousand transactions is as visible as a hundred. The committed series remains exactly $\{1, \dots, S_m\}$, so suppression cannot be disguised as message loss: a lost receipt would leave a hole, and no hole ever appears. The counter is per organization — 30 receipts from a second organization alongside 50 from the first yield independent, individually gap-free series — so one organization’s omission cannot be masked by another’s volume.

We then test backfilling, the natural evasion once a shortfall is challenged. After 100 honest receipts the bank anchors 10 more carrying processedAt timestamps inside the window already covered. The series stays gap free and S_m rises to 110, so a count-only check would now balance. But every backfilled receipt holds a sequence number above receipts bearing later timestamps, and comparing the two orders flags exactly 10 inversions — so the specific insertions are identified, not merely their number.

Chaincode-side execution costs 36–39 μs per receipt, so sequence assignment adds no meaningful cost to the write path. We state the scope of this result plainly: it validates the completeness logic and the gap-free invariant, not the deployed system’s end-to-end commit latency or its behaviour under MVCC contention. Those are properties of the live three-organization network. **[[Report measured commit latency (p50/p95/p99) and sustained receipt throughput per organization from blockchain/benchmark/once the network run is recorded.]]**

E. Why not an append only database

A PostgreSQL audit log with append only triggers is simpler and faster, but its guarantee is conditional on the honesty of whoever administers it — in AML, the bank under scrutiny, whose administrator can update rows, disable triggers, and truncate write ahead logs. The single-logger constructions of Section II-B narrow this gap but still leave the log in the custody of the audited party. On Fabric the regulator holds an independent replica, block validation requires a majority across three organizations, and (2) lets the regulator bound what it has not been shown without issuing a challenge. The converse holds honestly: for a single institution whose regulator trusts its internal controls and where cross-institution sharing

TABLE II
Comparative performance on the Elliptic test set (9,643 transactions, 503 illicit), temporal split, threshold 0.5

Model	Acc. (%)	Prec. (%)	Recall (%)	F_1 (%)	ROC-AUC	MCC	Train (s)	TN	FP	FN	TP
LightGBM	96.52	67.08	65.21	66.13	0.9633	0.643	5.24	8979	161	175	328
Random Forest	97.29	87.12	56.46	68.51	0.9605	0.689	27.14	9098	42	219	284
XGBoost	96.36	66.52	60.84	63.55	0.9555	0.617	11.94	8986	154	197	306

^aAccuracy and MCC are derived from the confusion counts using (4). Training times measured on the same hardware.

TABLE III
Completeness reconciliation, $N = 1000$ transactions per run

k	S_m	Δ_m	Detected	Gap-free	$\mu\text{s}/\text{receipt}$
0	1000	0	—	yes	38.9
1	999	1	yes	yes	37.2
10	990	10	yes	yes	36.6
100	900	100	yes	yes	35.8

^a k receipts suppressed by the bank; S_m read by the regulator from its own replica; $\Delta_m = N - S_m$. Gap-free means the committed series is exactly $\{1, \dots, S_m\}$.

is not required, a hash-chained database log suffices and blockchain is unjustified overhead.

VII. Limitations and Conclusion

Four limitations bound these results. The models are trained on Bitcoin features, and although the adaptation of Section efsec:features covers all six FATF categories, 47 of 85 positions are zero padded in bank deployment, so the reported metrics characterize Elliptic rather than wire transfers. The sequence counter serializes an organization’s receipt writes, bounding throughput. Completeness proves count consistency, establishing that records are missing without identifying which; binding receipts to the bank’s internal transaction numbering would pin the specific omission, at the cost of a durable ordered anchor queue. Finally, the argument depends on N_m being obtained independently of the bank, converting the problem of trusting the audit trail into one of trusting the volume return — easier, but not vacuous.

This paper argued that anchoring fraud alerts on a blockchain does not make an AML programme accountable, because the decision to raise an alert is itself the lever a dishonest institution would pull. We presented a system in which every scoring decision produces an immutable receipt whose series completeness is enforced by chaincode assigned per organization sequence numbers written atomically with each receipt, so a missing record becomes an arithmetic shortfall computable from the regulator’s own peer. Detecting omission from a log is long studied; what is new is enforcing it inside multi-party consensus, where the verifier is a consortium member and the ledger must withhold business content from a competitor — achieved through keyed pseudonymization and a private data collection scoped to the bank and regulator. On Elliptic under a temporal split, LightGBM achieved 65.21% recall at 0.9633 ROC-AUC and was selected over the higher precision Random Forest on cost asymmetry grounds. Future work follows the limita-

tions: sharding the counter to lift the throughput ceiling, restoring aggregated neighbourhood features to close the detection gap, a periodic Merkle commitment over the bank’s daily transaction log, and retraining on institution specific wire transfers.

References

- [1] B. Oztas, D. Cetinkaya, F. Adedoyin, M. Budka, G. Aksu, and H. Dogan, “Transaction monitoring in anti-money laundering: A qualitative analysis and points of view from industry,” *Future Generation Computer Systems*, vol. 159, pp. 161–171, 2024.
- [2] M. Weber, G. Domeniconi, J. Chen, D. K. I. Weidele, C. Bellei, T. Robinson, and C. E. Leiserson, “Anti-money laundering in Bitcoin: Experimenting with graph convolutional networks for financial forensics,” in *KDD Workshop on Anomaly Detection in Finance*, 2019, arXiv:1908.02591.
- [3] Y. Elmougy and L. Liu, “Demystifying fraudulent transactions and illicit nodes in the Bitcoin network for financial forensics,” in *Proc. 29th ACM SIGKDD Conf. Knowledge Discovery and Data Mining*, 2023.
- [4] A. Pareja, G. Domeniconi, J. Chen, T. Ma, T. Suzumura, H. Kanezashi, T. Kaler, T. B. Schardl, and C. E. Leiserson, “EvolveGCN: Evolving graph convolutional networks for dynamic graphs,” in *Proc. 34th AAAI Conf. Artificial Intelligence*, 2020, pp. 5363–5370.
- [5] G. Guardiola Muzquiz and E. Soriano Salvador, “SealFSv2: Combining storage-based and ratcheting for tamper-evident logging,” *International Journal of Information Security*, vol. 22, no. 2, pp. 447–466, 2023.
- [6] S. A. Crosby and D. S. Wallach, “Efficient data structures for tamper-evident logging,” in *Proc. 18th USENIX Security Symposium*, 2009, pp. 317–334.
- [7] H. Kang, T. Dai, N. Jean-Louis, S. Tao, and X. Gu, “FabZK: Supporting privacy-preserving, auditable smart contracts in Hyperledger Fabric,” in *49th IEEE/IFIP Int. Conf. Dependable Systems and Networks*, 2019, pp. 543–555.
- [8] A. A. Khan, A. Alsufyani, N. Alsufyani et al., “BAML: A decentralized approach to secure, privacy-preserving financial compliance for enhancing anti-money laundering with blockchain hyperledger and federated learning,” *Peer-to-Peer Networking and Applications*, vol. 18, art. 270, 2025.
- [9] E. Androulaki, A. Barger, V. Bortnikov et al., “Hyperledger Fabric: A distributed operating system for permissioned blockchains,” in *Proc. Thirteenth EuroSys Conf.*, 2018, pp. 1–15.
- [10] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [11] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
- [12] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, “LightGBM: A highly efficient gradient boosting decision tree,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 3146–3154.
- [13] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, “SMOTE: Synthetic minority over-sampling technique,” *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.
- [14] S. M. Lundberg and S.-I. Lee, “A unified approach to interpreting model predictions,” in *Advances in Neural Information Processing Systems*, vol. 30, 2017, pp. 4765–4774.
- [15] D. Chicco and G. Jurman, “The advantages of the Matthews correlation coefficient (MCC) over F1 score and accuracy in binary classification evaluation,” *BMC Genomics*, vol. 21, art. 6, 2020.
- [16] Financial Action Task Force, “International standards on combating money laundering and the financing of terrorism and proliferation: The FATF Recommendations,” FATF, Paris, 2023.